Java compilation and execution

Java Compilation & Execution Process (Method 1 Detailed)

Overview: Two-Stage Process

Java uses a **two-stage execution model**:

```
Source Code (.java) → [Compiler] → Bytecode (.class) → [JVM Interpreter] → Machine Code → Output
```

Stage 1: Compilation with `javac`

What Happens:

```
javac HelloWorld.java
```

Output: `HelloWorld.class` (the compiled bytecode file)

What is a `.class` File?

A `.class` file is **not machine code** — it's **platform-independent bytecode**:

```
HelloWorld.java (Source)          HelloWorld.class (Bytecode)
┌──────────────────────┐          ┌──────────────────────┐
│ public class Hello   │    →     │ CA FE BA BE (magic)  │
│ {                    │          │ Version info         │
│   public static ...  │          │ Constant pool        │
│ }                    │          │ Methods (bytecode)   │
└──────────────────────┘          │ Fields               │
                                  │ Attributes           │
                                  └──────────────────────┘
```

Key points:

- `CA FE BA BE` = Magic number identifying it as Java bytecode
- Contains **intermediate instructions** (not CPU instructions)
- **Same `.class` runs on any OS** (Windows, Linux, Mac) if JVM is installed
- ~3-4x smaller than source code

What `javac` Does:

1. **Lexical Analysis** — Breaks source into tokens

```
public class HelloWorld { ... }
↓
KEYWORD(public) CLASS IDENTIFIER(HelloWorld) BRACE_OPEN ...
```

2. **Syntax Checking** — Validates grammar

```
if (x = 5)   // ERROR: assignment in condition
if (x == 5) // OK
```

3. **Type Checking** — Ensures type safety

```
String name = 123;   // ERROR: cannot assign int to String
```

4. **Bytecode Generation** — Creates `.class` file with JVM instructions

```
public static void main(String[] args) {
    System.out.println("Hello");
}
↓ (compiled to bytecode)
LDC "Hello"
INVOKEVIRTUAL java/io/PrintStream.println
RETURN
```

Stage 2: Execution with JVM (Java Virtual Machine)

What Happens:

```
java HelloWorld
```

The JVM reads `HelloWorld.class` and executes it

The JVM Process:

```
1. Class Loading
   ↓
2. Bytecode Verification (linking & validation)
   ↓
3. Interpretation/JIT Compilation
   ↓
4. Execution on CPU
```

LINKING - The Critical Phase ⚠️

Linking is done by the JVM at RUNTIME (when you run `java`), not by `javac`

What is Linking?

Linking = Resolving **references between classes and libraries**

Example:

```
// HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello");  // Reference to System & PrintStream classes
    }
}
```

When `javac` compiles this, it creates:

```
Method: main
  Instructions:
    - Load constant "Hello"
    - Call method: java/lang/System.out.println(String)
    - RETURN
```

But at compile time, `javac` DOESN'T check if `System.out.println()` actually exists — it just records the reference.

Summary with Example

Complete Example: Step-by-Step

File: `HelloWorld.java`

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Step 1: Compile

```
$ javac HelloWorld.java
```

What happens:

- `javac` reads `HelloWorld.java`
- Checks syntax ✓
- Checks types ✓
- Generates `HelloWorld.class` (bytecode file, ~400 bytes)
- Writes to disk

File created: `HelloWorld.class`

```
CA FE BA BE 00 00 00 34 00 1D 07 00 02 01 00 0A
48 65 6C 6C 6F 57 6F 72 6C 64 07 00 03 01 00 10
...binary bytecode...
```

Step 2: Run

```
$ java HelloWorld
```

What happens:

Phase	Details
Class Loading	JVM finds `HelloWorld.class` in current directory
Linking	JVM checks if `System` class exists in JDK (resolves external refs)
Verification	JVM validates bytecode safety (no buffer overflows, etc)
Execution	JVM interprets/compiles bytecode → prints "Hello, World!"

Output:

```
Hello, World!
```

Memory management

General concept that applies to all applications (Whether you write code in **C++, C#, Python, JavaScript, or Java**)
When, compiled or run on modern operating systems (like Windows, macOS, or Linux), the operating system will always allocate these exact core segments to your application's **Process**.

Think of RAM (computer's brain (**RAM**)) as a giant warehouse. To avoid total chaos, the computer neatly partitions this warehouse into specialized sections for each task (**Process**) given to it.
And inside that section, it organizes its data inside. This is called Memory management.

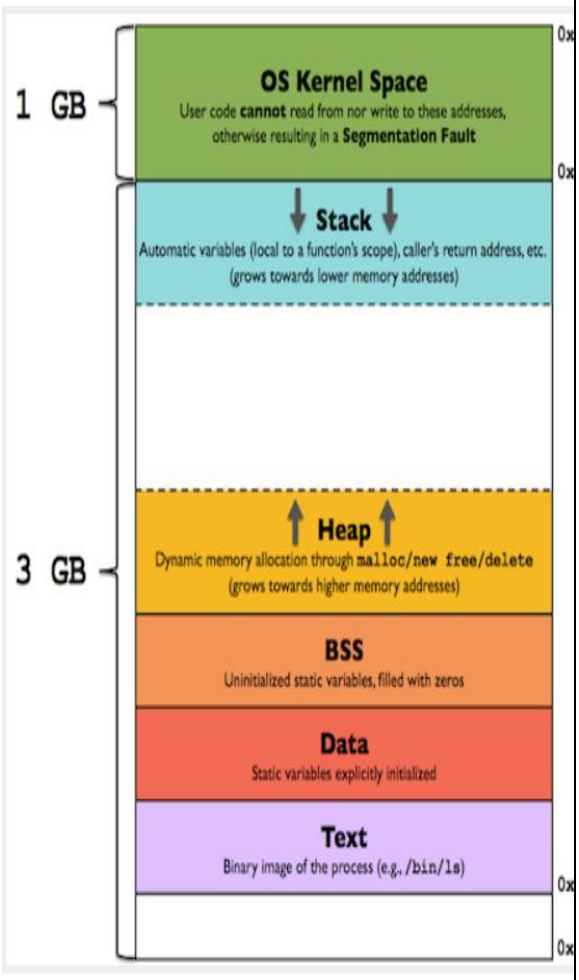
Video link:
Memory management in Java: https://www.youtube.com/watch?v=aAjkJW08BGQ
Constructor: youtube.com/watch?reload=9&v=Hpg4fBTCYuA
Stack Vs Heap allocation: geeksforgeeks.org/dsa/stack-vs-heap-memory-allocation/
Read: https://notionalmachines.github.io/nms/StackAndHeapDiagram-3.html

```
-------------------------------------------------------------------------

                                OPERATING SYSTEM PROCESS (The Sandbox

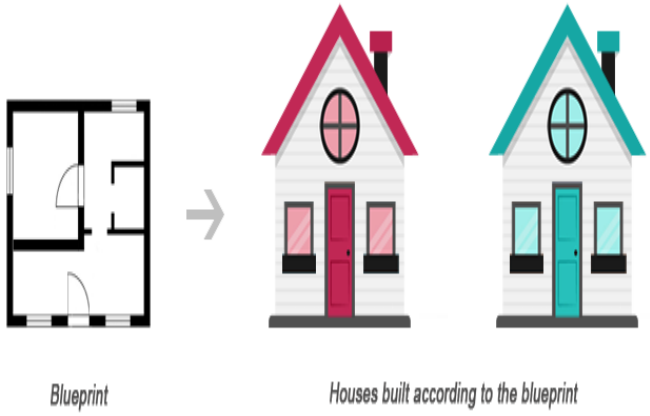
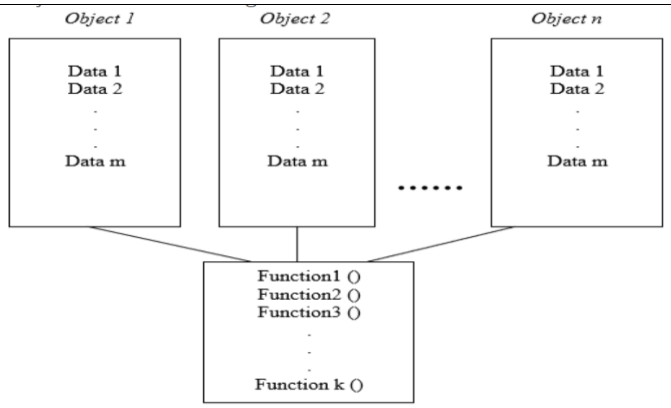
[ SHARED MEMORY AREAS - Shared by all Tasks & Threads ]
+-------------------------+  +-----------------------------------------
|     1. TEXT SEGMENT     |  |                        2. DATA SEGMENT
| (Compiled Bytecode:     |  | (Class Fields & Statics:
|  Class blueprints)      |  |  `studentCount` static variable live here
+-------------------------+  +-----------------------------------------
|                                    3. THE HEAP SPACE
|
|  +---------------------------+  +---------------------------+  +------
|  |   String Constant Pool    |  |  Object Area ('new')      |  |
|  |   `"Alex"` literal string |  |  `SchoolDemo` Instance    |  | (Mana
|  +---------------------------+  +---------------------------+  +------
+-------------------------------------------------------------------------

[ PRIVATE MEMORY AREAS - Unique to each Task execution thread ]
+---------------------------------------+      +----------------------
|       4A. STACK (Main Thread)         |      |       4B. STACK (Execu
|                                       |      |
| [ Frame: processData() ]              |      | [ Frame: run() Task ]
| - `scoreParam` = 10                   |      | - `workerLocalCalc` =
|                                       |      |
| [ Frame: main() ]                     |      |
| - `grade` = 10                        |      |
| - `test` = (Pointer to Heap Object)   |      |
+---------------------------------------+      +----------------------
-------------------------------------------------------------------------
```



VSCode – Run Java main

When you click "Run" on a java program, the Operating System creates “Process Sandbox” with 4 memory segment and loads the .class (ByteCode, platform independent instructions for CPU) to “Text Segment”. Java Virtual Machine JVM

Text Segment	Contains the compiled bytecode of your program—the actual instructions that the Java Virtual Machine (JVM) executes.
Class is in text segment. Memory is not allocated until Main creates object from Blue print.	
Primitive Data Types vs. Classes In programming, primitive data types are the basic building blocks that hold simple, single values, while classes are user-defined blueprints used to create complex objects that bundle both data and behavior. Think of primitive data types as **raw materials** (like wood, bricks, or nails) and classes as **blueprints for a house** (which specify how to combine those materials and what the house can actually do).	
When a class is specified, 1> memory space of all the member function is allocated – in Text segment. but memory allocation is not done for the data member. 2> When an object is created, memory allocation for its data members is done. Same member functions but they may be storing different data in their data members.	

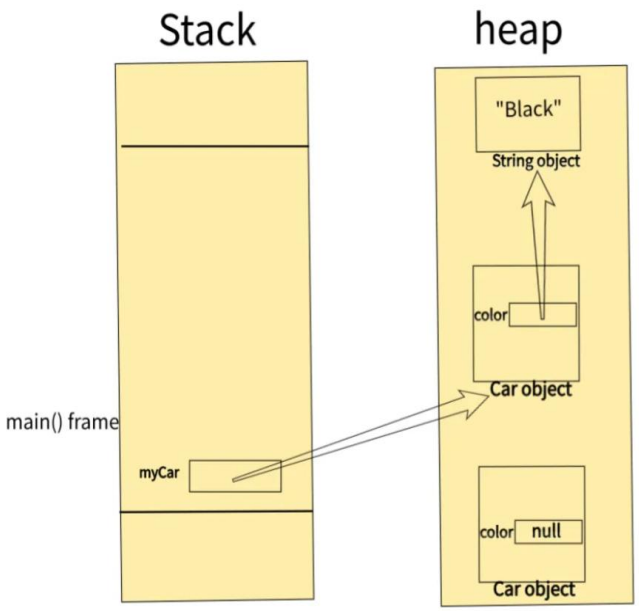
``
class Car {
    public String color;
}
 	
public class Main {
   public static void main(String[] args) {
        // Anew Car object Allocated on the heap, but its reference is not preserved! (not accesible/referenced) Garbage collection eligible):
      new Car();
``
	In Java, most of your day-to-day coding interaction happens dynamically between the **Stack** and the

```
// Reference type Varaiable (allocated on the stack) (pointing to null)
        Car myCar;


// New Car object Allocated on the heap and its reference stored on myCar on the stack.
        myCar = new Car();

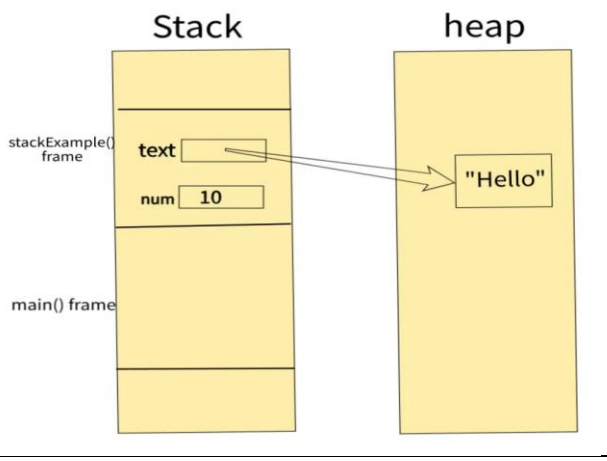
// New String object allocated in heap and its reference is stored in instance reference variabl color of the object
       mycar.color = "Black";
   }
}
```

Heap.




```
public class Example {
    public void stackExample() {
// Stored in the stack
    int num = 10;

// Reference is in the stack, but "Hello" (object of class String) is in Heap (in the string pool area)
    String text = "Hello";
 }
}
```



Stack

The Stack is crucial for managing method execution, storing method call frames, local variables, and references to objects in the heap. Each thread has its own stack, providing thread isolation and supporting concurrent execution.

Characteristics of the Stack

- **Local Variables and References**
- **Function parameter**
- **Maintains methods call stack**
- **LIFO Memory Management**: The stack follows a Last-In-First-Out (LIFO) approach. Each method call adds a stack frame, and the frame is removed when the method completes.

Heap

The Heap is the main memory pool for dynamic allocation in Java, storing all objects created during runtime. Java's automatic garbage collection handles memory cleanup when objects are no longer referenced, making the heap essential for long-term storage.

Characteristics of the Heap:

Instance Variables and Object Allocation: When you create an object using new, it's allocated on the heap and reference stored in Stack.

Data segment

```
public class MemoryExample {
    // These are stored in the Data Segment (static memory)
    static int staticCount = 0;          // Allocated at class loading
    static String staticName = "Java";   // Stored in Data Segment
    static final double PI = 3.14;       // Constant in Data Segment

    public void incrementCount() {
        staticCount++;  // Modifies the value in Data Segment
    }
}
```

Key Points:
- Immutable string literals are pooled to save memory
- Static variables exist throughout program lifetime
Variables created for whole Class (not instance)
- All instances share static variables
- Data Segment is allocated once at class loading

static variables and **string literals** that exist for the entire program lifetime.

```
public class StringLiteralExample {
    static String str1 = "Hello";    // String literal in Data Segment
    static String str2 = "Hello";    // Same reference (String pool)

    public static void main(String[] args) {
        String str3 = "Hello";       // Points to same String pool entry
        String str4 = new String("Hello"); // Creates new object in Heap

        System.out.println(str1 == str2);     // true (same pool reference)
        System.out.println(str1 == str3);     // true (same pool reference)
        System.out.println(str1 == str4);     // false (different objects)
        System.out.println(str1.equals(str4)); // true (same content)
    }
}
```

Garbage collection

Once objects lose references (e.g., after a variable goes out of scope), they become eligible for garbage collection.

```
public class GarbageCollectionExample {
    static List<String> staticList = new ArrayList<>();  // Data Segment

    public static void main(String[] args) {
        // Scenario 1: Object eligible for GC
        String tempName = new String("John");
        // Stack: tempName ref
        // Heap: "John" object

        tempName = null;  // Clear the reference
        // Heap: "John" object is now unreachable
        // GC will collect it in next cycle

        // Scenario 2: Object in ArrayList (not GC'd)
        Person p1 = new Person("Alice", 30);
        staticList.add(p1.getName());
        p1 = null;
```

Memory Leak

```
// Static collection holds references indefinitely
static List<MyObject> cache = new ArrayList<>();

void addToCache(MyObject obj) {
    cache.add(obj);  // Added to static list
}
// Objects never removed, never GC'd
// Result: Gradual memory leak
```

However, the way these segments are managed internally differs depending on the programming language. Here is a direct comparison between **Java** and standard compiled languages like **C/C++**:
Memory Segments Across Different Languages

Memory Segment	C / C++ (Native Applications)	Java (Virtual Machine Applications)
Text Segment	**Native Machine Code:** Holds raw binary instructions directly executed by your computer's CPU.	**Bytecode:** Holds intermediate Java instructions executed by the JVM interpreter/JIT compiler.
Data Segment	**Global/Static Variables:** Allocated by the OS at startup. Lives until the app closes.	**JVM Metaspace/Heap:** Statics are managed inside a special subsystem of the JVM rather than raw OS globals.
Stack Segment	**Manual/Automated Stack:** Handled directly by the CPU stack pointer. Highly efficient.	**JVM Stacks:** Each Java thread gets a private stack managed by the JVM software layer, though it maps closely to OS memory layout.
Heap Segment	**Manual Heap:** The developer must explicitly allocate (`malloc`) and manually free (`free`) the memory. **If you forget, you get a memory leak.**	**Managed Heap:** The **Garbage Collector** scans the heap automatically. You use `new` to allocate, but you never manually delete objects.

Why Java is Unique

In standard native applications (like a video game written in **C++**), your app talks **directly to the Operating System**.
In **Java**, there is a middleman. When you start a Java application, the OS actually launches the **Java Virtual Machine (JVM) process**. The JVM pretends to be its own little computer inside your real computer. The JVM takes the broad Text, Data, Heap, and Stack blocks given to it by the OS and manages them via its own interna l software structures to provide helpful safety nets like automatic **Garbage Collection**.

Primitive Data Types vs. Classes

While a Class allows you to build complex, multi-featured objects using a blueprint, a **Primitive** is like a single raw material (like a loose brick or a nail). YouTube · Java Easily +1

Feature	Primitive Data Types (The Bricks)	Classes / Objects (The Blueprint & Houses)
What it is	A single, raw, fundamental value built directly into the language.	A complex, custom template (blueprint) that combines data and actions.
Examples	`int` (integers), `double` (decimals), `char` (letters), `boolean` (true/false).	`String` , `Scanner` , or a custom `House` class you write yourself.
Capabilities	**Data only.** It cannot perform actions or hold multiple types of info.	**Data + Actions.** It holds data (variables) and things it can do (methods).
Memory	Small and simple. Stores the actual value directly in memory.	Larger. The variable stores a reference (a pointer) to where the object lives.
Syntax Clue	Usually written in all **lowercase** (e.g., `int` , `char`).	Usually capitalized in standard code naming conventions (e.g., `String` , `House`).

Visualizing Inheritance

Inheritance is organized hierarchically, moving from a general concept down to specific variations. The diagram below illustrates how a general parent class passes its core structure down to specialized child classes:

Understanding Inheritance

Inheritance is a concept in programming where one class adopts the attributes (variables) and behaviors (methods) of another class. It works exactly like **family inheritance** or **biological classification**.

Instead of writing the same code over and over again for similar objects, you create a general class first, and then create specific versions of it.

🏢 The Parent and Child Relationship

- **Superclass (Parent Class):** The general class that shares its features.
- **Subclass (Child Class):** The specialized class that inherits those features and can add its own unique spin.

🚗 Real-World Example: Vehicles

Imagine you are coding a video game with different types of vehicles.

1. **The Parent Class (`Vehicle`):** Every vehicle has a `speed` and a `fuelType`. Every vehicle can `accelerate()` and `brake()`.
2. **The Child Class (`ElectricCar`):** Instead of rewriting `speed`, `fuelType`, `accelerate()`, and `brake()`, the `ElectricCar` simply **inherits** them from `Vehicle`.
3. **The Unique Twist:** The `ElectricCar` can then add its own unique property, like `batteryLevel`, and a unique action, like `chargeBattery()`.

💡 Why use Inheritance?

- **Code Reusability:** You write the code once in the parent class, and all child classes get it automatically.
- **Organization:** It keeps your code neat, logical, and easy to update. If you want to change how all vehicles `brake()`, you only have to fix it in one place (the parent class).

When you create an object from a child class (a subclass), computer memory handles it very specifically: **every single object gets its own completely separate, isolated copy of the parent's data variables.**

If you change the variable inside one child object, it **will not affect** the other child object.

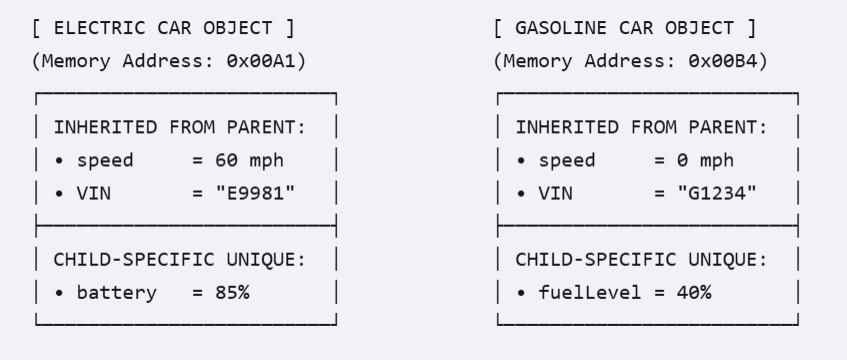
The Blueprint vs. Isolated Objects

Think of a class as a template. When you use that template to create child objects, the computer carves out totally separate spots in memory for each one:
Imagine you have a parent class called Vehicle which has a variable called speed. Then you create two distinct child objects from it: car1 and car2.

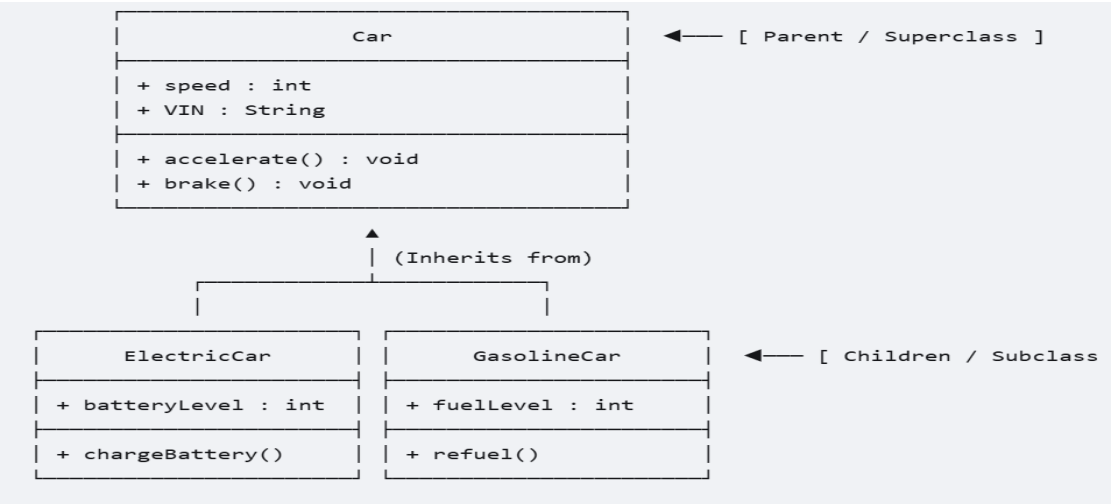
1. **Object 1 (car1):** Gets its own variable chunk for speed. If you accelerate car1 to **60 mph**, only its personal variable changes.
2. **Object 2 (car2):** Gets a completely separate variable chunk for speed. It stays at **0 mph** even though car1 is speeding away.

How the Data Copies Work in Memory

When you actually create real objects in your code using these blueprints, the computer carves out **completely separate, isolated boxes in memory** for each individual vehicle.
Each child object inherits the foundational structures from the Car blueprint, but fills those boxes with its own completely unique values:



Here is the official **Class Diagram** for your car hierarchy.
In a standard computer science class diagram, the arrows point **upward from the child to the parent** to represent the relationship *"is a"* (e.g., an Electric Car *is a* Car).



This shows how an OS **Process** hosts **Shared Blocks** (Text, Data, Heap) alongside **Private Worker Threads** (Stacks).

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class SchoolDemo {
    // DATA SEGMENT: Created at class startup, accessible globally.
    public static int studentCount = 500;

    public static void main(String[] args) {
        // STACK (Main Thread): Raw value stored inside main's frame
        int grade = 10;

        // HEAP (String Pool): "Alex" goes to the pool; `name` holds its address
        String name = "Alex";

        // HEAP: 'new' operator allocates permanent physical object space
        SchoolDemo test = new SchoolDemo();

        // HEAP: The executor manager object is built here
        ExecutorService manager = Executors.newFixedThreadPool(1);

        // A separate background task is assigned to an independent thread
        manager.submit(() -> {
            // PRIVATE STACK (Worker Thread): Completely isolated variable frame
            int workerLocalCalc = grade * 2;
            System.out.println("Worker calculated: " + workerLocalCalc);
        });

        test.processData(grade);
        manager.shutdown();
    }

    public void processData(int scoreParam) { // Temporary stack frame pushed ove
        int adjustments = scoreParam + 5;
    } // Frame pops off and dissolves immediately right here
}
```

```java
class Person {
    int id;
    String name;

    public Person(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

public class PersonBuilder {
    private static Person buildPerson(int id, String name) {
        return new Person(id, name);
    }

    public static void main(String[] args) {
        int id = 23;
        String name = "John";
        Person person = null;
        person = buildPerson(id, name);
    }
}
```

Let's analyze this step-by-step:

1. When we enter the *main()* method, a space in stack memory is created to store primitives and references of this method.
   - Stack memory directly stores the primitive value of the integer *id.*
   - The reference variable *person* of type *Person* will also be created in stack memory, initially pointing to null and later updated to point to the actual object in the heap.
2. The *main* method is calling the *buildPerson()* static method, for which allocation will take place in stack memory on top of the previous one.
3. The *buildPerson()* calls the parameterized constructor *Person(int, String)* which will allocate further memory on top of the previous stack. This will store:
   - The *this* object reference of the calling object in stack memory
   - The primitive value *id* in the stack memory
   - The reference variable of *String* argument *name,* which will point to the actual string from string pool in heap memory
4. However, heap memory will store all instance variables for the newly created object *person* of type *Person.*

Let's look at this allocation in the diagram below:

